

Du notebook au projet livrable en 8 étapes



Le script perso qui tourne chez toi, transformé en projet présentable, avec l'erreur fréquente à corriger et le résultat visible à chaque étape.

Tu as un notebook qui marche. Ce n'est pas la même chose qu'un projet qu'un recruteur peut ouvrir et comprendre en deux minutes. Ce guide déroule les 8 étapes concrètes qui séparent le script perso du lien GitHub que tu peux coller dans une candidature.

Chez Datafuji, on relit beaucoup de premiers portfolios de juniors. Le même écart revient presque à chaque fois : le code tourne très bien en local, sur la machine de son auteur, et devient illisible dès qu'un recruteur clone le repo. Pas parce que l'analyse est mauvaise. Parce que rien n'a été pensé pour un lecteur externe.

Les 8 étapes ci-dessous ne rendent pas ton analyse meilleure. Elles rendent visible le travail que tu as déjà fait.

Poser les fondations du repo.

Avant même de lire une ligne de code, le recruteur juge l'organisation du dossier.

1. Étape 1 : structurer le repo ERREUR FRÉQUENTE

Cas : Le classique du junior : un dossier avec `analyse_finale_v3.ipynb`, `ventes.csv` et `export.png` à plat, à la racine, mélangés avec le README.

Erreur fréquente

Tout en vrac à la racine. Le recruteur ouvre le repo, voit 15 fichiers sans dossier, et referme l'onglet avant d'avoir lu une ligne de code.

Résultat visible

Un repo lisible en 5 secondes, avant même d'ouvrir un fichier.

```
projet-ventes/  
  data/  
    raw/  
    processed/  
  notebooks/  
    01_exploration.ipynb  
  src/  
    clean.py  
    build_report.py  
  outputs/  
    figures/  
  README.md  
  requirements.txt
```

2. Étape 2 : écrire un vrai README ERREUR FRÉQUENTE

Cas : Beaucoup de repos juniors ont un README généré par défaut : un titre Markdown suivi d'une ligne vide. Aucune information utile.

Erreur fréquente

Un README vide ou générique. Le recruteur doit ouvrir le code pour comprendre le sujet, ce qu'il ne fera pas.

Résultat visible

Le recruteur comprend le projet sans ouvrir une seule cellule de notebook.

```
## Contexte
Analyse des ventes 2024 d'un e-commerce fictif : identifier
les 3 catégories qui expliquent la baisse du T3.

## Installation
pip install -r requirements.txt

## Usage
python src/build_report.py

## Résultat
outputs/figures/baisse_t3.png : la catégorie Mobilier
explique 42% de la baisse du chiffre d'affaires au T3.
```

3. Étape 3 : figer les dépendances ERREUR FRÉQUENTE

Cas : Le notebook utilise **pandas** et **matplotlib** installés à un moment donné, sans version notée nulle part.

Erreur fréquente

Aucune trace des bibliothèques utilisées. Le recruteur clone le repo, lance le script, et `ModuleNotFoundError` après `ModuleNotFoundError`.

Résultat visible

Le projet s'installe et tourne sur une autre machine, sans erreur.

```
# requirements.txt
pandas==2.2.2
matplotlib==3.8.4
python-dotenv==1.0.1

# génération depuis ton environnement
pip freeze > requirements.txt
```

À retenir

Un repo qui se comprend avant même d'être lancé.

Structure, README, dépendances : les trois étapes qui coûtent le moins de temps et qui filtrent le plus de candidatures.
Un recruteur qui ne comprend pas ton repo en 30 secondes passe au suivant.

Rendre le code lisible.

Ici, le recruteur teste si un inconnu peut suivre ton raisonnement sans toi à côté.

4. Étape 4 : nommer pour un lecteur externe ERREUR FRÉQUENTE

Cas : Un script nommé `df2_final_vraiment.py` qui appelle une fonction `f()` sur une variable `x2`. Toi, tu sais ce que c'est. Un inconnu, non.

Erreur fréquente

Des noms qui n'ont de sens que pour toi, au moment où tu les as écrits. Six mois plus tard, tu ne les comprends plus non plus.

Résultat visible

Un inconnu suit le fil du code sans commentaire ni note orale de ta part.

```
# avant
def f(x2):
    return x2.groupby('c').sum()

# apres
def total_ventes_par_categorie(ventes: pd.DataFrame) -> pd.DataFrame:
    return ventes.groupby("categorie")["montant"].sum()
```

5. Étape 5 : gérer les données proprement ERREUR FRÉQUENTE

Cas : Un script qui commence par `pd.read_csv("C:/Users/dylan/Bureau/ventes.csv")`, un chemin Windows codé en dur. Ça tourne chez toi, nulle part ailleurs.

Erreur fréquente

Un chemin absolu en dur, et des données brutes commitées directement dans Git sans distinction entre brut et transformé.

Résultat visible

Le pipeline tourne sur une autre machine sans toucher une ligne de code.

```
# .gitignore
data/raw/*.csv
data/processed/*.csv

# code : chemin relatif au projet, jamais un chemin absolu
ventes = pd.read_csv("data/raw/ventes.csv")
```

Prouver que ça marche.

Ici, le recruteur regarde si tu contrôles ton résultat, ou si tu espères qu'il est bon.

6. Étape 6 : ajouter des tests légers ERREUR FRÉQUENTE

Cas : Une fusion de deux tables qui, silencieusement, duplique des lignes à cause d'une clé non unique. Le script tourne, le chiffre est faux.

Erreur fréquente

Zéro vérification après transformation. Le pipeline continue même quand le résultat n'a plus de sens.

Résultat visible

Le pipeline s'arrête et te prévient avant de produire un chiffre faux.

```
# sanity checks a placer juste apres la transformation
assert ventes_clean.shape[0] > 0, "table vide apres nettoyage"
assert ventes_clean["montant"].min() >= 0, "montant negatif detecte"
assert ventes_clean["categorie"].notna().all(), "categorie manquante"
```

7. Étape 7 : produire une visualisation finale ERREUR FRÉQUENTE

Cas : Le graphique de debug laissé tel quel : `plt.plot(x, y)` sans titre, sans légende, avec un nom d'axe brut.

Erreur fréquente

Un graphique compréhensible seulement par toi, qui as le contexte en tête. Sans toi à côté, il ne dit rien.

Résultat visible

Un message clair et lisible sans avoir besoin d'ouvrir le notebook.

```
fig, ax = plt.subplots()
ax.bar(baisse["categorie"], baisse["part_pct"])
ax.set_title("Mobilier explique 42% de la baisse du CA au T3")
ax.set_ylabel("Part de la baisse (%)")
fig.savefig("outputs/figures/baisse_t3.png", dpi=150)
```

À retenir

Un projet qui prouve son résultat sans toi à côté.

Un sanity check ne rend pas ton analyse plus intelligente, il empêche un bug silencieux de devenir un chiffre faux publié.

Un graphique titré ne rend pas ton résultat plus vrai, il le rend lisible sans explication orale.

Publier.

La dernière étape n'est pas technique : c'est celle qui rend le travail réellement consultable.

8. Étape 8 : publier sur GitHub ERREUR FRÉQUENTE

Cas : Un historique de commits avec un seul message, `final version 2` vraiment. Aucune trace de la progression du travail.

Erreur fréquente

Un repo privé, ou un historique d'un seul commit fourre-tout, qui ne montre ni méthode ni progression.

Résultat visible

Un lien GitHub prêt à coller dans une candidature, sans rien à corriger avant.

```
# checklist avant de rendre le repo public
git log --oneline
# 5f2a1c9 ajoute la visualisation finale
# 8b0d3ee ajoute les sanity checks
# 1a44f21 nettoie les chemins et le .gitignore
# c02e9aa premiere structure du repo

# repo public, README a jour, requirements.txt present
```

Les 8 étapes, du script au lien que tu peux envoyer : structurer le repo, écrire un vrai README, figer les dépendances, nommer pour un lecteur externe, gérer les données proprement, ajouter des sanity checks, produire une visualisation qui parle seule, publier avec un historique propre. Aucune de ces étapes ne change ton analyse. Toutes changent si quelqu'un d'autre peut la comprendre.

Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquer un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

